Department of Computer Science, University College London

COMP0222/COMP0249 Lab 08: Lidar Odometry and Point Cloud Mapping

3rd March, 2026 ver. 20260303

1 Scope and Purpose

In this lab, you will gain hands-on experience with a 2D scanning laser rangefinder, the RPLIDAR A2M12. You will implement a driver to visualize raw sensor data and subsequently develop a Scan-to-Map algorithm to build a 2D point cloud representation of an unknown environment. By physically moving with the sensor, you will explore the principles of laser odometry, the Iterative Closest Point (ICP) algorithm, and the practical challenges of real-time Simultaneous Localization and Mapping (SLAM).

2 Scenario

Motion in a 2D plane is characterized by a rigid body transformation consisting of translation (x, y) and rotation (θ). This forms the robot's pose vector: $\mathbf{x} = [x, y, \theta]^T$. In this lab, you will perform Laser Odometry. Unlike wheel odometry, which counts wheel revolutions and slips on loose terrain, laser odometry estimates motion by comparing what the robot sees now to what it saw a moment ago. By aligning the current laser scan with a cumulative point cloud (the map), you can estimate the sensor's change in pose. This technique is fundamental for autonomous service robots, such as vacuum cleaners and warehouse automated guided vehicles (AGVs).

3 Sensor Specification: RPLIDAR A2M12

The RPLIDAR A2 is a 360-degree Laser Range Scanner (LIDAR), see Fig. ??. It operates based on the principle of Laser Triangulation.

- **Mechanism: The system emits a modulated infrared laser signal. The laser reflection is captured by a vision acquisition system. By analyzing the position of the focused spot on the image sensor, the distance to the object is calculated.**
- **Rotation: The rangefinder core rotates clockwise to perform a 360-degree omnidirectional scan.**
- **Key Specifications:**
  - **Distance Range: 0.2m – 12m**
  - **Angular Resolution:** $\approx 0.25^\circ$
  - **Sample Rate: Up to 8000 - 16000 samples/second**
  - **Scan Rate: 5-15Hz, Typical 10Hz**
  - **Connection: USB (UART to USB converter included)**

Figure 1: RPLIDAR A2M12 and its componenets.

4 Requirements

This lab requires Python 3.8+. You must install the following libraries. If you are using NumPy 2.0 or newer, you may encounter compatibility issues with `scikit-learn` **or** `scipy`**. ICP relies on Nearest Neighbors for point correspondence, and** `scikit-learn` **is used for establishing the correspondences. It is recommended to pin** `NumPy` **to version 1.26.x if errors occur.**

```
pip install rplidar-roboticia pygame matplotlib scikit-learn
# Optional compatibility fix if using older SciPy versions:
# pip install "numpy<2", or to be specific pip install numpy==1.26.4
```

5 Activities

Activity 0: Basic Connectivity and Visualization

- **Objective: Establish communication with the sensor and visualize the raw polar coordinate data (r, θ) converted into Cartesian space (x, y).**
- **Instructions:**
  - **Connect the RPLIDAR to your computer via USB.**
  - **Identify the `COM` port (Windows) or `/dev/tty.usb*` path (Linux/Mac). If needed, you will need to change the port number.**
    - **In Windows, you will find the port number under `Device Manger/Ports (COM & LPT)`.**
    - **In Linux/Mac, run `ls /dev/tty.usb*`, to find the port number.**
    - **If needed, install the driver, `CP210x Universal Windows Driver`, from here: https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads**
  - **Run the following Python script. This script handles the coordinate transformation from Polar to Cartesian and renders the points in real-time.**
- **Code: `rplidar_viewer.py`.**

Once the code runs successfully, you will see the scan as shown in Fig. ??.

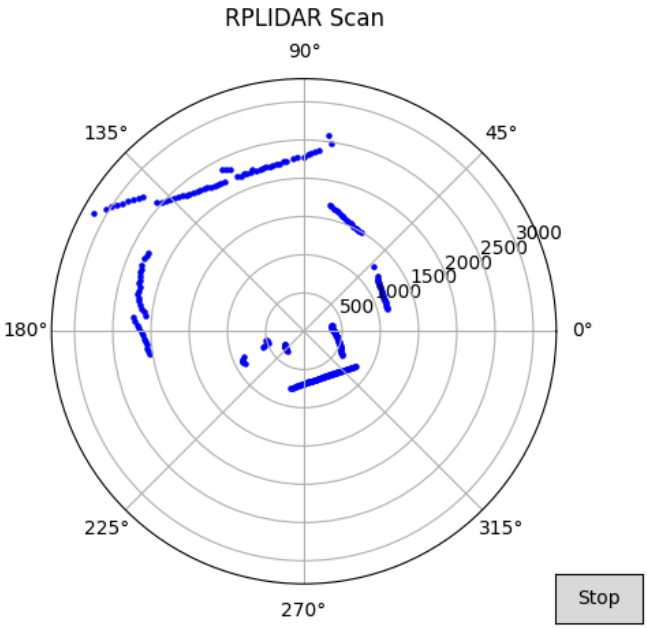

Figure 2: RPLIDAR A2M12 scan viewer.

Activity 1: Converting Range/Bearing Measurements

- **Objective: The sensor provides raw data in a polar coordinate system (Range ρ, Bearing θ). Your objective is to convert these into a Cartesian system (x, y) so they can be plotted on a 2D map.**
- **Instructions:**
  1. **Open the file** `rplidar_plotter.py` **in your preferred editor.**
  2. **Locate the section marked:** `# Activity 1: Converting Range/Bearing Measurements`.
  3. **Implement the conversion logic using the formulas below.**
- **Hint: Use the Python** `math` **library for trigonometric functions. Note:** `math.sin()` **and** `math.cos()` **expect angles in radians.**

$$x = \rho\cos(\theta),\ \ y = \rho\sin(\theta)$$

  Where ρ is the range and θ is the bearing.
- **Outcome: Upon completion, run the script. You should see the scan points visualized, see Fig. ??.**

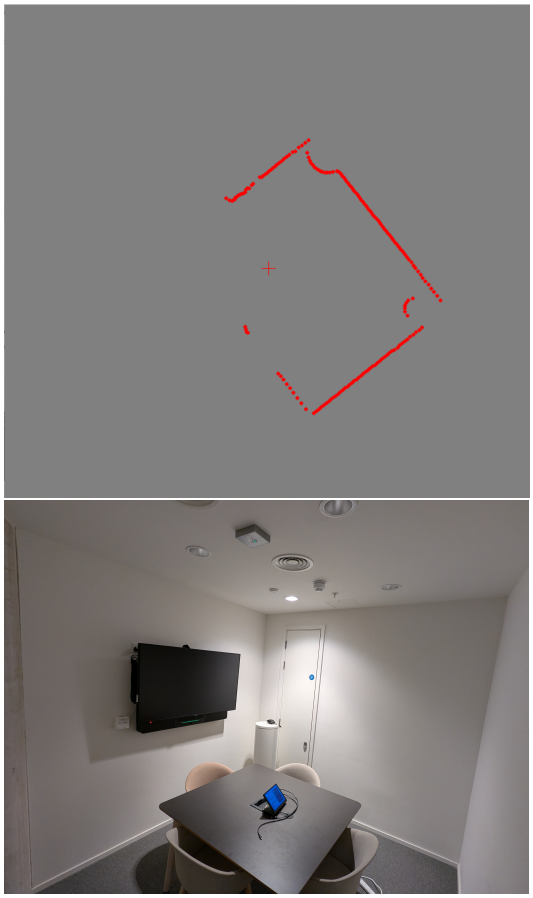

Figure 3: (top) RPLIDAR A2M12 scan plotter, scanning a room. (bottom) The view of the room.

Activity 2: Scan-to-Map Matching

- **Objective: Implement a simplified SLAM (Simultaneous Localization and Mapping) system. There is no loop closure, no map management, e.g., feature pruning. You will use the Iterative Closest Point (ICP) algorithm to align the current laser scan with a "local map" built from previous scans. Please refer to the Appendix before completing the rest of the steps.**
- **When holding the sensor and walking, your body occupies a portion of the scan (usually behind the sensor). Because SLAM algorithms assume the environment is static, your moving body will appear as a "moving wall," causing the map to distort or the robot to"slide" incorrectly. We will implement an Angle Filter to create a "Blind Spot" behind the sensor (swath of $270°$), effectively ignoring your body.**
- **Instructions:**
  - **Open the file `rplidar_icp.py` in your preferred editor.**
  - **Locate the section marked: `# Activity 2: Calculate cross term`.**
  - **Refer to Eq. (??) to complete the activity.**
  - **Run the code.**
  - **Hold the Lidar with the USB cable pointing towards you.**
  - **Walk slowly through a room.**
  - **Observe how the incoming scan (red) aligns with the map (Grey).**
  - **Viewer controls:**
    - `W/A/S/D`**: Move the camera.**
    - `Q / E`**: Zoom In / Out.**
    - `SPACE`**: Camera view reset.**
    - `ESC`**: Quit.**
    - `R`**: Reset.**
- **Tips for Running the Algorithm:**
  - **Start Slowly: The scan matching assumes you don't teleport. Move the Lidar gently.**
  - **Environment: It works best in a room with corners and walls. It struggles in long, smooth hallways (nothing to "lock" onto) or open spaces.**
  - **Reset: If the map gets messy (which happens if you move too fast and the matching breaks), close the window and restart. Real SLAM systems have "Loop Closure" to fix this, but this is a pure odometry script.**
  - **Avoid Roll and Pitch: The algorithm assumes that the sensor's motion is constrained to a 2D plane (3 Degrees of Freedom: x, y, θ). Consequently, only rotation about the vertical axis (Yaw) is valid. Ensure you hold the sensor level. Any tilting of the sensor (Roll or Pitch) will cause the laser to scan the floor or ceiling, introducing geometric errors into the map.**
  - **Cart: You may put the sensor on a cart or a chair with wheels to ensure the sensor does not experience roll or pitch motions.**
  - **Discussion Question: How can we handle sensor motion that is not constrained to 2D? If roll and pitch are unavoidable (e.g., a handheld device or a drone), what additional hardware (e.g., IMU) or algorithmic steps would be needed to correct the laser scan data?**
  - **Outcome: Upon completion of the activity, you should see the map and trajectory, as shown in Fig. ??.**

Figure 4: (top) Map (gray) and the trajectory (blue), generated by RPLIDAR A2M12 scan (red), scanning a room. The green circle shows the current location of the sensor. (bottom) The view of the room.

Activity 3: Environmental Challenges and Degeneracy

Objective: To evaluate the limitations of the scan-matching algorithm by testing it in environments with varying geometric properties.

Part A: Feature-Rich Environment (The Baseline)

1. **Setup: Stand in a corner of the lab or a cluttered area with visible obstacles (legs of tables, boxes, corners).**
2. **Action: Move the sensor slowly in various directions.**
3. **Question: Does the map remain stable? Why do corners help the ICP algorithm lock the position?**

Part B: The Corridor Problem (Geometric Degeneracy)

1. **Setup: Move to a long, straight corridor with smooth walls.**
2. **Action: Walk steadily down the center of the hallway.**
3. **Observation: You may notice the estimated position "stalling" or sliding unpredictably along the path of travel.**
4. **Explanation: This is a case of Geometric Degeneracy. In a featureless corridor, the laser sees two parallel lines. Mathematically, sliding the scan forward (along the corridor axis) results in zero error change. The algorithm lacks sufficient constraints to determine forward motion.**

Part C: Loop Closure and Drift

1. **Setup: Walk a large loop (e.g., a 10m x 10m square path) and return to your exact starting position.**
2. **Observation: Check if the map "closes" correctly. Do the walls at the end align with the walls from the start?**
3. **Explanation: You will likely observe Drift. Small angular errors accumulate over time (integration error). Over a long path, a 1° error can result in a significant positional offset, causing a "double wall" effect upon return.**

Activity 4: Dynamic Limits

Objective: To experimentally determine the maximum translational and rotational velocities the system can track before localization failure occurs.

1. **Translation Test:**
   - **Start by walking at a slow pace (≈ 0.5 m/s).**
   - **Gradually increase your speed to a brisk walk.**
   - **Observation: Note the speed at which the map begins to "tear" or double. This occurs when the displacement between consecutive frames exceeds the convergence basin of the ICP algorithm.**
2. **Rotation Test:**

- **Stand in a fixed position and rotate the sensor 90 degrees slowly.**
- **Reset, then rotate the sensor quickly (e.g., $> 90°$/sec).**
- **Observation: Rapid rotation often causes catastrophic failure. Unlike translation, small angular errors cause large displacements for distant points ($ArcLength = r \times \theta$), making rotational tracking significantly more challenging.**

Activity 5 (optional): Lidar Data Acquisition & Replay

Objective: In this activity, you will interface with the RPLidar A2M12 sensor. Instead of writing raw serial commands, you will use a Driver Class that handles the hardware connection, operating system detection, and file logging for you. This is an optional lab, but having it will help you to run the next activity efficiently. File `lidar_driver.py` in your project folder contains a class that is the "engine" for data collection and reply.

- **Part 1: Recording Data**
  - **Goal: Capture a 10-second scan of your environment.**
  - **Connect the RPLidar to your USB port. Run the file `rplidar_recorder_example.py` from terminal. It will save lidar scan to file `lab_data_01.json`.**
- **Part 2: Replay - Data Verification**
  - **Goal: Verify that the file you wanted was created and contains valid data without needing the robot connected.**
  - **Run the file `rplidar_loader_example1.py`. You should see the data printing to the terminal exactly as if the robot were running, but "simulated" from your file.**
  - **Run the file `rplidar_loader_example2.py`. You should see the data is visulised this time.**
- **Hints:**
  - `mode='live'`: **Connects to hardware, streams data, writes to file.**
  - `mode='replay'`: **Connects to file, streams data, no hardware needed.**
  - **Data Format: The driver provides a list of tuples: (quality, angle, distance).**

Activity 6 (optional): Parameter Sensitivity Analysis

Objective: To explore the trade-offs between computational cost, sensor range, and mapping accuracy.

1. **Lidar Range (`MAX_RANGE_MM`):**
   - **Experiment: Reduce the maximum range from 4000mm to 1000mm. Also try increasing it.**
   - **Question: How does this affect stability in a large room? Does the "Corridor Problem" (degeneracy) happen more frequently when the sensor cannot see the end of the room?**
2. **Solver Iterations (`ICP_MAX_ITER`):**
   - **Experiment A (Low Precision): Set iterations to 1.**
   - **Observation: Does the map drift or "lag" behind your movements?**
   - **Experiment B (High Precision): Set iterations to 50.**

- **Observation: Notice the impact on the frame rate (FPS). Is the increased accuracy worth the computational cost, or does the delay actually make tracking worse?**

To perform a rigorous and scientific comparison of algorithm performance, you need to decouple data collection from processing. You can do this by using the previous module, record data, and process them with different parameters, each time running the algorithm.

A System Description: Point-to-Plane ICP Formulation

This appendix details the Point-to-Plane Iterative Closest Point (ICP) algorithm. For this derivation, we assume that all point data and rigid body transformations (rotation and translation) are constrained to a 2D plane.

Consider two point cloud sets:

- **Source:** $\mathcal{S} = \{\mathbf{s}_i\}_{i=1}^{N}$ **(The current scan to be transformed)**
- **Destination:** $\mathcal{D} = \{\mathbf{d}_i\}_{i=1}^{M}$ **(The reference map or target cloud)**

The objective is to determine the optimal rigid transformation T (comprising a rotation R and translation t) that aligns the Source set to the Destination set.

Unlike standard Point-to-Point ICP, which minimizes the Euclidean distance between point pairs, the Point-to-Plane variant minimizes the perpendicular distance between the source point and the local surface geometry (tangent line) of the destination point. This approach is generally more robust to sliding errors along flat walls, which is common in indoor environments.

A.1 Geometric Intuition

Let $\mathbf{s}_i$ be a point in the source cloud and $\mathbf{d}_i$ be the corresponding point in the destination cloud. We seek a rotation matrix R and a translation vector t that transforms the source point to a new position $\mathbf{s}'_i$, such that:

$$\mathbf{s}'_i = \mathbf{R}\mathbf{s}_i + \mathbf{t} \tag{1}$$

In the Point-to-Plane variant, we do not minimize the standard Euclidean distance $\|\mathbf{s}'_i - \mathbf{d}_i\|$. Instead, we minimize the projection of that distance onto the surface normal vector $\mathbf{n}_i$ at the destination point.Mathematically, this minimizes the squared dot product of the error vector and the normal (Fig. ??):

$$\min_{\mathrm{R,t}} \sum_i \left((\mathbf{s}'_i - \mathbf{d}_i) \cdot \mathbf{n}_i\right)^2 \tag{2}$$

Geometrically, this allows the source points to "slide" along flat surfaces (like walls) without penalty, provided they align correctly with the surface orientation.

B The Error Metric

Let s_i be a source point, d_i the corresponding target point, and n_i the normal at d_i.
The transformed source point is:

$$s'_i = \mathbf{R}s_i + \mathbf{t} \tag{3}$$

The error function to minimize is the sum of squared dot products:

$$E = \sum_i \left((\mathbf{R}s_i + \mathbf{t} - d_i) \cdot n_i\right)^2 \tag{4}$$

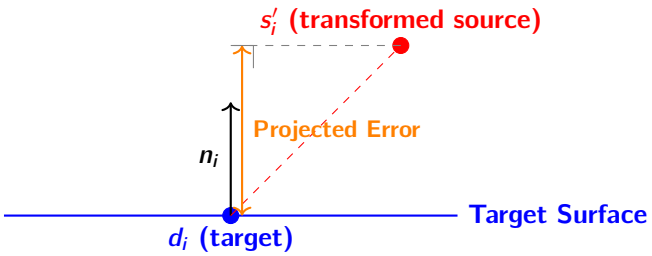

Figure 5: Point-to-Plane ICP.

- **This equation is non-linear due to the rotation matrix R (contains $\cos\theta, \sin\theta$).**
- **Unlike Point-to-Point, there is no closed-form SVD solution for this specific metric.**
- **Solution: We must linearize.**

C Linearizing the Rotation

Assumption: If the rotation angle θ is small, we can approximate:

$$\cos(\theta) \approx 1, \quad \sin(\theta) \approx \theta \tag{5}$$

The 2D Rotation matrix becomes:

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \approx \begin{bmatrix} 1 & -\theta \\ \theta & 1 \end{bmatrix} \tag{6}$$

Applying this to $s_i = [s_{ix}, s_{iy}]^T$:

$$\mathbf{R}s_i \approx \begin{bmatrix} s_{ix} - \theta s_{iy} \\ \theta s_{ix} + s_{iy} \end{bmatrix} \tag{7}$$

The optimization becomes a linear system of equations $\mathbf{Ax} = \mathbf{b}$, where we solve for $x = [\theta, t_x, t_y]^T$.

D Estimating Normals

In real scans, we don't know the surface function. We use Principal Component Analysis (PCA) on local neighbors.

1. **Gather Neighbors: Find k nearest neighbors for point p.**
2. **Covariance Matrix:**
$$C = \frac{1}{k}\sum_{j=1}^{k}(p_j - \bar{p})(p_j - \bar{p})^T \tag{8}$$
3. **Eigen Decomposition: Solve $Cv = \lambda v$.**
4. **Select Normal: The eigenvector corresponding to the smallest eigenvalue represents the direction of least variance (the normal).**

Note: PCA normals have ambiguous sign ($\pm$). We must enforce consistency (e.g., flip if pointing away from viewpoint).

E Point-to-Plane Algorithm Steps

1. **Preprocessing: Estimate normals n_i for the target point cloud (using PCA).**
2. **Loop until convergence:**
   (a) **Match: Find closest point d_i in Target for each s_i in Source.**
   (b) **Construct System: For each pair, set up the linear equation:**

$$\theta(s_{ix}n_{iy} - s_{iy}n_{ix}) + t_x n_{ix} + t_y n_{iy} = (d_i - s_i) \cdot n_i \tag{9}$$

   For all m points, this becomes:

$$\begin{bmatrix} (s_{1x}n_{1y} - s_{1y}n_{1x}) & n_{1x} & n_{1y} \\ \vdots & \vdots & \vdots \\ (s_{ix}n_{iy} - s_{iy}n_{ix}) & n_{ix} & n_{iy} \\ \vdots & \vdots & \vdots \\ (s_{mx}n_{my} - s_{my}n_{mx}) & n_{mx} & n_{my} \end{bmatrix} \begin{bmatrix} \theta \\ t_x \\ t_y \end{bmatrix} = \begin{bmatrix} (d_1 - s_1) \cdot n_1 \\ \vdots \\ (d_i - s_i) \cdot n_i \\ \vdots \\ (d_m - s_m) \cdot n_m \end{bmatrix} \tag{10}$$

   which is in this form $Ax = b$, and can be solved easily.
   (c) **Solve: Use Least Squares to find $x = \begin{bmatrix} \theta \\ t_x \\ t_y \end{bmatrix}$.**
   (d) **Update: Apply exact rigid transform to Source.**